

# **Отчёт по лабораторной работе №13**

**Программирование в командном процессоре ОС UNIX. Ветвления  
и циклы**

Магос Иоаннис

# Содержание

|          |                                       |          |
|----------|---------------------------------------|----------|
| <b>1</b> | <b>Цель работы</b>                    | <b>5</b> |
| <b>2</b> | <b>Выполнение лабораторной работы</b> | <b>6</b> |
| <b>3</b> | <b>Вывод</b>                          | <b>9</b> |
| 3.1      | Контрольные вопросы . . . . .         | 9        |

## Список иллюстраций

|     |                            |   |
|-----|----------------------------|---|
| 2.1 | Первый скрипт . . . . .    | 6 |
| 2.2 | второй скрипт . . . . .    | 7 |
| 2.3 | третий скрипт . . . . .    | 7 |
| 2.4 | четвёртый скрипт . . . . . | 8 |

## **Список таблиц**

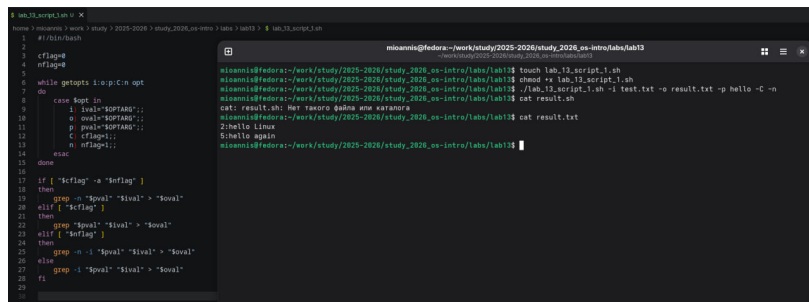
# 1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

## 2 Выполнение лабораторной работы

1. Выполняем первую задачу: Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами:

- `-iinputfile` — прочитать данные из указанного файла;
- `-ooutputfile` — вывести данные в указанный файл;
- `-р`шаблон — указать шаблон для поиска;
- `-C` — различать большие и малые буквы;
- `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-р`.



```
1 #!/bin/bash
2
3 cflag=
4 nflag=
5
6 while getopts i:op:C:n opt
7 do
8     case $opt in
9         i) iinput="OPTARG";;
10        o) oout="OPTARG";;
11        p) pval="OPTARG";;
12        C) cflag=1;;
13        n) nflag=1;;
14        *)
15            echo "Invalid option"
16            exit 1
17        ;;
18    esac
19 done
20
21 if [ "$cflag" = "1" ]
22 then
23     grep -n "$pval" "$iinput" > "$oout"
24 else
25     grep -n "$pval" "$iinput" > "$oout"
26 fi
27
28 if [ "$nflag" = "1" ]
29 then
30     grep -n "$pval" "$iinput" > "$oout"
31 else
32     grep -n "$pval" "$iinput" > "$oout"
33 fi
34
35 exit 0
```

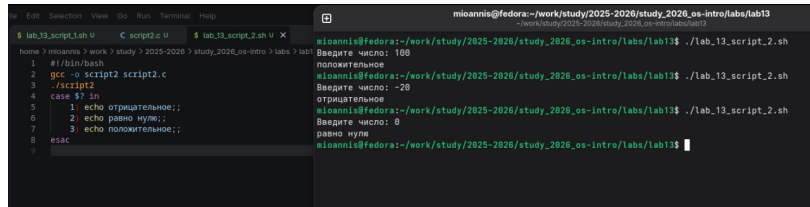
```
micannia@fedora:~/work/study/2025-2026/study_2026_os_intro/labs/lab13$ touch lab_13_script.1.sh
micannia@fedora:~/work/study/2025-2026/study_2026_os_intro/labs/lab13$ chmod +x lab_13_script.1.sh
micannia@fedora:~/work/study/2025-2026/study_2026_os_intro/labs/lab13$ ./lab_13_script.1.sh -i test.txt -o result.txt -p hello -C -n
cat result.txt
2:hello Linux
5:hello again
micannia@fedora:~/work/study/2025-2026/study_2026_os_intro/labs/lab13$
```

Рисунок 2.1: Первый скрипт

2. Выполняем вторую задачу:

- Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию

в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.

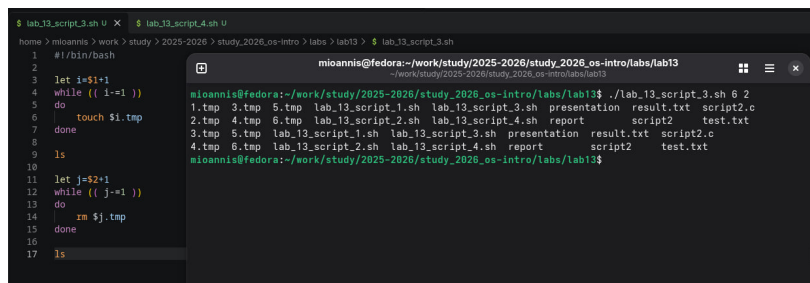


```
lab13_script1.sh  C script2.c  lab13_script2.sh  X
home > mioannis > work > study > 2025-2026 > study_2026_os-intro > labs > lab13
1 #!/bin/bash
2 gcc -o script2 script2.c
3 ./script2
4 case $? in
5     1) echo отрицательное;;
6     2) echo равно нулю;;
7     3) echo положительное;;
8 esac
9
mioannis@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab13$ ./lab13_script2.sh
Введите число: 100
Положительное
mioannis@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab13$ ./lab13_script2.sh
Введите число: -20
Отрицательное
mioannis@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab13$ ./lab13_script2.sh
Введите число: 0
Равно нулю
mioannis@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab13$
```

Рисунок 2.2: второй скрипт

### 3. Выполняем третью задачу:

- Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до `N` (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).

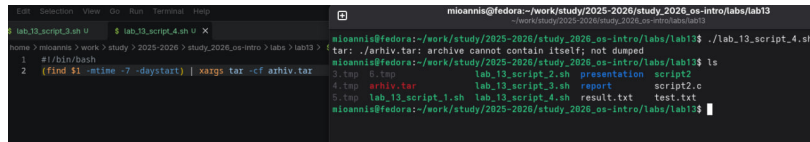


```
lab13_script3.sh  X  lab13_script4.sh  U
home > mioannis > work > study > 2025-2026 > study_2026_os-intro > labs > lab13 > $ lab13_script3.sh
1 #!/bin/bash
2
3 let i=$1-1
4 while (( i >= 0 ))
5 do
6     touch $i.tmp
7 done
8
9 ls
10
11 let j=$2-1
12 while (( j >= 0 ))
13 do
14     rm $j.tmp
15 done
16
17 ls
18
mioannis@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab13$ ./lab13_script3.sh 6 2
1.tmp 3.tmp 5.tmp lab13_script1.sh lab13_script3.sh presentation result.txt script2.c
2.tmp 4.tmp 6.tmp lab13_script2.sh lab13_script4.sh report script2 test.txt
3.tmp 5.tmp lab13_script1.sh lab13_script3.sh presentation result.txt script2.c
4.tmp 6.tmp lab13_script2.sh lab13_script4.sh report script2 test.txt
mioannis@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab13$
```

Рисунок 2.3: третий скрипт

### 4. Выполняем четвёртую задачу:

- Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`).



```
micannis@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab13$ tar: ./arhiv.tar: archive cannot contain itself; not dumped
micannis@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab13$ ls
0.tmp  0.tmp      lab_13_script_2.sh  presentation  script2
4.tmp  arhiv.tar   lab_13_script_3.sh  report       script2.c
5.tmp  lab_13_script_1.sh  lab_13_script_4.sh  result.txt   test.txt
micannis@fedora:~/work/study/2025-2026/study_2026_os-intro/labs/lab13$
```

Рисунок 2.4: четвёртый скрипт

## 3 Вывод

В данной работе мы изучили основы программирования в оболочке ОС UNIX и писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

### 3.1 Контрольные вопросы

1. Каково предназначение команды `getopts`? Ответ: Создание по пользовательским аргументам.
2. Какое отношение метасимволы имеют к генерации имён файлов? Ответ: Используют как файлы так и аргументы.
3. Какие операторы управления действиями вы знаете? Ответ: `If`, `else`, `elif`, `fi`, `while`, `do`, `done`, `until`, `do`, `done`, `for`, `in`, `do`, `done`, `case`, `in`, `esac`
4. Какие операторы используются для прерывания цикла? Ответ:
  - `for` – будет выполнять действие до тех пор, пока есть объекты для выполнения.
  - `while` – выполняет действие до тех пор, пока условие является истинным.
  - `until` – будет выполняться пока условие не станет правдиво.

5. Для чего нужны команды false и true? Ответ: until – будет выполняться до тех пор, пока условие не станет true, т.е. пока оно не станет false.
6. Что означает строка `if test -f mans/i.$s`, встреченная в командном файле? Ответ: Проверяет если существует файл его размерность и тип с двумя разными расширениями, заменяя через переменные.
7. Объясните различия между конструкциями while и until. Ответ:
- while – выполняет действие до тех пор, пока условие является истинным.
  - until – будет выполняться до тех пор, пока условие не станет истинным, т.е. пока оно false.